1 Event responses and the emitted product

This section covers two duties of the network estimator of the Network-mode document: the response to events on carried state (a slip, a tracking gap, a satellite's absence and return, a restart of the wind-up model, a gross code fault on one line of sight), and the emitted product (which clock and phase-bias values reach the user, how a change is declared, how the user answers). The carried local column, the bounded drop operator, the held S-basis and the coupling time update belong to the float-ambiguity-vehicle section. Settings are quoted with their loader defaults; the reference configurations are `config/golden/golden_field_refound.json` (real data) and `golden_sim_refound.json` (simulated network), which agree on every setting named here.

1.1 The event doctrine

A carried quantity enters the next epoch as a prior. In the information form of the between-epoch carry document, a carried value $\hat{x}$ with variance P folds as the pseudo-observation

$$x = \hat{x} + e, \qquad e \sim \mathcal{N}\big(0,\ P + q\,\Delta t\big), \tag{1}$$

with q the process-noise rate of its kind. If the quantity moved by δ after $\hat{x}$ was formed, the pseudo-observation is biased by δ at weight $(P + q\,\Delta t)^{-1}$. For a time-linked quantity $q \approx 0$ and P is small, so the states sharing its rows (ionosphere, clocks, troposphere) absorb the bias; a wired response must change $(\hat{x}, P)$ before the fold. The project rule is that a mechanism changing which information flows into which states is an event at both edges. The decoupling edge bounds what the decoupled states can accumulate; the re-coupling edge re-validates or re-seeds what it reconnects. Each edge writes an event row and a counter, so an inert response reads as zero.

Vocabulary. The parameter registry (`network/assembly/ParamRegistry`, `network/assembly/Responses`) names five events (`ParamEvent`: slip, gap, handover, decouple, readmit) and seventeen policies (`ResponsePolicy`). Those used here: *Drop* returns the variance to birth strength, clears the border regression and the block row, and keeps the value (the float-ambiguity-vehicle section); *EraseAndRebirth* erases the entry for the next harvest to re-create; *Park*, *SeedAtVarPlusQGap* and *Discard* keep an entry past its column's departure, seed the returned column, or drop the entry unfolded; *Hold* freezes and re-applies an entry. Each response site is one of 68 rows of `ResponseSite`, a (kind, event, policy, code site) tuple with an arming predicate (`responseSiteArmed`) and a counter; a site that responds where it is not declared armed is a run-end declaration error. Marks that only re-seed a linearisation point are excluded, among them the pending-reseed mark, which PASS B's measurement pre-pass consumes (a solve no-op on the float vehicle). The acting ambiguity response is the free-epoch latch (`network.cycle_slip.relevel_free_epoch`), which ends in a Drop. A flagged satellite clock whose reinitialisation sigma is not positive is left out of the stash entirely (a hard refloat); the sites `ClkHardRefloat`, `CbHardRefloat` and `PbHardRefloat` count each entry left out, and are inert on the reference configurations, where the clock monitor does not run.

Armed set. The reference configurations arm the MW/LLI and IF triple-difference slip drains (`network.cycle_slip.mw_detect`, `if_phase_td`; the network slip-QC section) with three refinements: `mw_evidence_refloat` admits an MW flag on its own strength (a lane without bias correction, an arc of at least `mw_evidence_min_arc_epochs` epochs, ratio to the threshold above `mw_`

evidence_ratio_min); common_band_scope frees only the bands the evidence implicates; anchor_slip_handover bans a freed phase anchor's family for that epoch, so the election picks a successor pinned at its own carried value. They also arm the responses of Section 1.2 to Section 1.10. Each arming setting is off in the loader.

1.2 One slip frees an arc once: the MW re-flag skip

An MW flag can re-detect a step the network has already answered. A locked arc absorbs a step and flags it samples later (drift demotion, then the unlocked flag), and the first sample after an LLI epoch flags the step that the LLI response freed at that epoch. The IF time-difference response and the gap re-entry response leave the same situation. Freeing the band a second time discards only the re-convergence since its re-level, and opens the decoupling edge twice for one event.

The MW detector reports the epoch t_s at which each flagged step entered the data ($t_s = 0$ when unknown): the flag's own epoch for a first detection, an earlier epoch otherwise. A band whose last re-level (LocalCarry::relevel_s, r_b) is not earlier than the step holds no pre-step level. With t the current epoch and $\tau = 0.5$ s (kMwReflagEpochTolS, half the shortest sampling interval),

$$\text{covered}(b) \;=\; \big(0 < t_s < t - \tau\big) \;\wedge\; \big(r_b > 0\big) \;\wedge\; \big(r_b \geq t_s - \tau\big). \tag{2}$$

Under network.cycle_slip.mw_reflag_skip the MW drain does not free a covered band; the candidate's other bands are freed as before, and the integer still reaches a waiting bridge pend, because the rule is tested after the delta-bridge intercept. A step of the current epoch is a first detection and is never covered. The slip-QC counters tally every such candidate and band whether or not the setting is armed (mw_reflag_*), and an armed skip writes an mw_wl_reflag event row with action skip naming the bands skipped and freed. The setting needs network.cycle_slip.mw_detect (the loader disarms it with a warning otherwise), is a forward-only response policy exempt from the snapshot fingerprint, and is armed on both reference configurations (docs/refound/MRF_mw_reflag_skip_2026-10-28.md).

1.3 Gap re-entry, keyed per signal

An arc that returns within T_{pl} (network.phase_leveling_max_gap_s, loader default 120 s, reference value 900 s) has its carried entry re-injected at the pre-gap value. No time-difference detector covers the return epoch: MW needs an adjacent predecessor on the same pair, the IF triple difference three consecutive epochs. The gap re-entry drains (networkSlipDrainGapReentry in network/slip/SlipQc) run serially in the PASS-A slip band, after the wind-up restart drain and before the inject, so an arc is freed in the epoch it returns.

Row-level verdict. networkComputeRawObservables builds one raw row per (station, satellite) on the two lowest distinct bands s_1, s_2 with phase and matching code. A row absent for at least one processed epoch and present now is a GapResumption, with verdict

$$\text{slipped} = \underbrace{\big(|\Delta\text{MW}| > g_{\text{MW}} \vee |\Delta\text{GF}| > g_{\text{GF}}\big)}_{\text{shifted}} \vee \big(\kappa_{\text{lli}} \wedge (\text{LLI}_{s_1} \vee \text{LLI}_{s_2})\big) \vee \big(\kappa_{\text{unm}} \wedge \neg\,\text{measurable}\big), \tag{3}$$

with $\Delta\text{MW}, \Delta\text{GF}$ the raw level changes across the break, g the station's empirical gates, κ_{lli} = gap_reentry_relevel_lli (default true) and κ_{unm} = gap_reentry_relevel_unmeasurable (default false: missing evidence is not evidence of a slip). A slipped row frees both bands.

The target is the carry. A returning arc has no live ambiguity column at drain time: it was pruned with the gap, and PASS B's pre-pass re-creates it after the drains. The response synthesises the inject's key (`StateKeys::makeAmbiguityKey`) and asks the carry whether anything needs protection: a leveling-carry entry (r, i, s) with

$$0 \le t - t_{\text{seen}} \le T_{\text{pl}}, \tag{4}$$

exactly what the inject would fold. A protectable arc enters the free-epoch latch (`relevel.free_ambs`, cause `kCauseGap`); a live column also takes the pending-reseed mark. `network.cycle_slip.relevel_free_epoch` is the latch: it carries the freed key past the pre-pass to the inject, where the arc folds at birth strength after the bounded drop of the float-ambiguity-vehicle section. An arc without a protectable entry is counted `no_carry`. The instrument `networkGapResumeCensus` (`network/slip/GapResumeCensus`) is the re-coupling count: it counts, for the arcs that returned after a break, those that re-entered on state that bridged the break, split by whether the raw level shifted, and those that re-entered as a fresh float. A funnel arc is never counted as bridged.

Per-signal key. The raw row is rebuilt every epoch from whichever phases have a matching code. A receiver that loses lock on its primary pair while another signal tracks through (band 5, or a fallback signal) keeps its row, so the row-level tier sees no gap and the returning phases would fold across the break. `network.cycle_slip.gap_reentry_per_signal` keys the re-entry on (station, satellite, signal). For a signal s last seen at epoch k_s, the level test differences raw phases (metres) against a partner p present at k and k_s:

$$d_s = \big[\Phi_s - \Phi_p\big](k) - \big[\Phi_s - \Phi_p\big](k_s) \approx \lambda_s\,\delta N_s - \lambda_p\,\delta N_p - (\mu_s - \mu_p)\,\delta I, \tag{5}$$

up to noise, multipath and the wind-up change. A partner that tracked through the gap is preferred: then $\delta N_p = 0$ and d_s isolates the candidate's step plus the ionospheric change over the gap. With g_{GF} the station's GF gate, which does not grow with the gap,

$$\text{slipped}_s = \big(|d_s| > g_{\text{GF}}\big) \vee \big(\kappa_{\text{lli}} \wedge \text{LLI}_s\big) \vee \big(\kappa'_{\text{unm}} \wedge \neg\,\text{measurable}_s\big), \tag{6}$$

with LLI_s the returning signal's own flag and κ'_{unm} = `gap_reentry_per_signal_unmeasurable` (default true: the carry of a satellite that stayed present is certain to fold). Only s is freed; the other bands tracked through. Both tiers share one responder (`gapReentryRespondArc`); the per-signal tier skips a satellite the row-level tier reported.

Co-resumed share. The level test of Equation (5) cannot see a step common to the candidate and its partner, which a channel-delay change at re-acquisition is. The signals of one (station, satellite) that return at the same epoch are one re-acquisition of that line of sight. With `network.cycle_slip.gap_reentry_coresumed_share` (default off, not armed on the reference configurations) the first co-resumed signal with a verdict of its own, shifted, LLI or unmeasurable, is the trigger, and every other co-resumed signal of the line of sight is freed with the trigger's cause (event cause `shared`, scope `los_shared`). A signal that tracked through the gap has no resumption and is never freed by the share, and an arc already in the free set of the epoch is not counted twice. The setting needs the per-signal tier and the row-level response; armed without them it is inert and the loader warns.

Tracker naming. The row-level tier named each band lexically: where the canonical band-1 phase is L1W the tracker named L1C, the key missed the carry, and the stale ambiguity folded. `network.cycle_slip.tracker_modeled_first` names each band by the station's canonical phase latch (`TrackerLatchLookup`) when that signal has raw phase and code this epoch, so the tracker key equals the carried key; `gap_tracker_key_mismatch` is then zero by construction.

1.4 Re-admission after a long absence

A satellite in eclipse, or below every mask, leaves the shared layout, and its clock and code-bias columns depart; on return they re-realise from code. `network.readmit_seed_horizon_s` $= T_{\mathrm{rd}}$ (loader default 0, off; reference value 3900 s) bridges the absence.

At departure the stash (`network/state/BorderStash`) parks each D2 clock entry whose column left the layout: it keeps the entry with its departure time t_{park}, unfolded. Code-bias entries park too under `network.readmit_seed_code_bias` (default true). Phase-bias entries do not: their frame is re-realised at the cold re-pin on return, and a stiff seed would conflict with that pin. When the column returns with $0 < t - t_{\mathrm{park}} \le T_{\mathrm{rd}}$, the fold decision (`carryFoldDecide` in `network/state/CarryDisposition`) is *Seeded*,

$$P_{\mathrm{seed}} = P + q\,(t - t_{\mathrm{park}}), \tag{7}$$

with q the registry rate of its kind. A clock under the temporal anchor (`network.clock_temporal_anchor`, armed) folds through the anchor with the staleness bound raised to $\max(\texttt{clock_anchor_max_gap_s}, T_{\mathrm{rd}})$ (`clockAnchorFoldVariance`), so it folds anchored over its true gap instead of re-drawing from code. The anchor's guard is the re-coupling check: a trip within its verify window (`kReadmitVerifyWindowS`, 300 s) after a seed is recorded as the data rejecting the pre-gap realisation (`readmit_seed_moved`) and decouples the clock. A stash-side check compares each seed with the solved value and counts a standardised residual above `kReadmitSeedK` (6).

A parked entry is dropped past the horizon (`ClkParkExpire`, `CbParkExpire`), when its column returns without consuming it (`ClkParkDiscard`), and at a clock-datum change of its constellation that the handover S-transform did not re-express (`ClkParkDiscardDatum`). The returning satellite is then a different arc and registers cold. The clock-only variant (`readmit_seed_code_bias` false) was falsified on the triple-frequency simulated network: the clock tails returned. The code-bias seeds fix the split between clock and code bias at the return; without them the cold code-bias realisation relocates into the clock (`docs/cards/network-vehicle/READMIT-EDGE.md`).

Seed mirrors. The epoch-tail snapshots that pair each folded border prior with its re-stashed solved value skip every parked entry, although the fold applies a seed at the variance P_{seed} of Equation (7). With `network.readmit_seed_mirrors` (armed on both reference configurations) a seed that the fold applied is a row of the carry variance-component monitor (code-bias family) and of the prior-residual diagnostic (`network/qc/PriorResidual`; code-bias and clock families), with the seed gap as its interval and the variance the fold used. The mirrors read the seed; they change no fold input unless the monitor's variance factor is applied (`network.carry_vce_apply`, off). The setting is inert without the horizon and without either reader, and the loader warns in both cases.

1.5 Carrier-phase wind-up: accumulation and its restart

The wind-up angle $\theta^{\mathrm{raw}} \in (-\pi, \pi]$ is an atan2 (the wind-up section of the observation-physics document), applied as $\lambda_s \theta / 2\pi$ on every phase signal of the satellite. In *fractional* mode (`estimation.accumulate_phase_windup` false, the loader default) $\theta = \theta^{\mathrm{raw}}$, and each crossing of $\pm\pi$ steps the

applied correction by one cycle on every signal inside a continuous arc. The reference configurations accumulate: `GnssCorrections::computePhaseWindUp` keeps (θ_{k-1}, t_{k-1}) per satellite and forms

$$\theta_k = \begin{cases} \theta_{k-1} + \mathrm{w}\left(\theta_k^{\mathrm{raw}} - \theta_{k-1}\right), & 0 < t_k - t_{k-1} < T_{\mathrm{wu}}, \\ \theta_k^{\mathrm{raw}} \quad \text{(restart)}, & \text{otherwise}, \end{cases} \qquad \mathrm{w}(x) = x - 2\pi\,\mathrm{round}(x/2\pi), \tag{8}$$

with T_{wu} = `estimation.phase_windup_max_gap_s` (loader default 300 s; not set by the reference configurations). A repeated call within one epoch ($t_k = t_{k-1}$) reuses the accumulated θ_k and is not a restart. A restart is recorded in the ledger `wuRestarts`. The state is keyed by satellite; each station owns its correction object, so the key is injective. The optional gap bridge (`estimation.phase_windup_gap_bridge`, default off, not armed) replaces the restart for gaps between T_{wu} and `estimation.phase_windup_bridge_max_gap_s` by a walk of the angle through virtual epochs; the walk and its refusals are described with the observable in the wind-up section of the observation-physics document, and a refused bridge falls back to the restart answered below.

The restart is an integer, band-common step. By (8), $\theta_k - \theta_k^{\mathrm{raw}} \in 2\pi\mathbb{Z}$ at every epoch, so the continuation a restart replaces equals $\theta_k^{\mathrm{raw}} + 2\pi n$ for an integer n. The restart moves the modelled phase of every signal by $-n\lambda_s$: the same integer number of cycles on every signal, which the float ambiguities see as a slip $\delta N_s = n$ for all s. The raw observations did not change, so no raw-observable detector can see it. In residual space

$$\frac{f_1 n\lambda_1 - f_2 n\lambda_2}{f_1 - f_2} = n\,\frac{c - c}{f_1 - f_2} = 0, \qquad c_1 n\lambda_1 - c_2 n\lambda_2 = n\,\frac{c}{f_1 + f_2} = n\,\lambda_{\mathrm{NL}}, \tag{9}$$

with c_1, c_2 the coefficients of the IF coefficients: the step is MW-blind and appears in the ionosphere-free combination as $n\lambda_{\mathrm{NL}} = n\lambda_{\mathrm{IF}}$. It also happens across a gap of at least T_{wu}, where no time difference has an adjacent predecessor. The leveling carry survives gaps up to T_{pl}, and $T_{\mathrm{wu}} < T_{\mathrm{pl}}$ on the reference configurations; a return after a gap in $[T_{\mathrm{wu}}, T_{\mathrm{pl}}]$ therefore steps the model while the carried ambiguity re-injects at its pre-gap value, the situation that every stiff time-linked quantity must have a wired response for.

The restart response. `network.cycle_slip.windup_restart_refloat` (disarmed by the loader without accumulation or phase leveling) drains each present station's ledger in the PASS-A slip band (`networkSlipDrainWindupRestart`). A record not stamped with the current epoch is refused as stale; a gap beyond T_{pl} is `beyond_carry`, since the carry was pruned with it. Otherwise `networkWindupRestartTargets` resolves the stepped arcs: the satellite's live ambiguity columns at the station if any exist, else its leveling-carry entries satisfying (4) that the arc ledger (`network/state/ArcLedger`) does not record as dead. Each target enters the free-epoch latch (cause `kCauseWindup`; the pending-reseed mark only on a live column), every target band is freed because the accumulator's verdict needs no raw corroboration, and the satellite's open bridge pends are voided, since a pend captured before the step would re-inject a value in the pre-restart frame. The drain must walk the carry and not only the live ambiguity map: a restarted arc's column was pruned with the gap, so a walk of live columns alone reaches no in-horizon restart, the inject folds the carry across the step, and the counter reports an answer that did not happen.

One mode for the network and the user. The measured gain of accumulation is in the user's own rows; the emitted product alone is neutral (`docs/refound/WUP_windup_accumulated_promotion_2026-10-01.md`). The S line of the emitted product declares both the wind-up construction (`windup_id` = `nominal-wu93`) and the mode (`wumode=`: `off`, `fractional` or `accumulated`,

from `windUpModeOf`); Section 1.9 describes the check. The in-process user inherits the network's mode through the derived configuration; a replay or stand-alone user holds the rule by configuration.

1.6 The gross line-of-sight screen

Post-fit RAIM (the network RAIM section) admits a row only if it carries an ambiguity coefficient above 0.5. A code row carries none, so a gross code error on one line of sight never enters $\hat{\sigma}_0$, a cell maximum or the median polish. `network.gross_los_screen` (`network/qc/GrossLos`) is the acting tier for code rows.

Statistic. Per station the seam forms $\boldsymbol{v} = \boldsymbol{y} - \boldsymbol{A}_s\hat{\boldsymbol{x}}_s - \boldsymbol{A}_r\hat{\boldsymbol{x}}_r$ with $\hat{\boldsymbol{x}}_r$ from the back-substitution of the Network-mode document. A code row touches no SH, ambiguity or `PB:` column, touches a `CB:` or `CLK:` column and names a satellite; coupling rows are skipped. The statistic

$$r_j = |v_j|\,\sqrt{w_j}\;f_j \tag{10}$$

uses the row's own weight w_j and undoes the deweight factor f_j the assembly applied (1 if none), so both bars read the data at nominal weight. The row's own σ is used because the pooled $\hat{\sigma}_0$ inflates while a station degrades, exactly when the screen must stay sensitive.

Declaration and release. For one (station, satellite) with n code rows, n_{over} of them above $k =$ `network.gross_los_k` (default 8), the epoch's evidence clears the bar when

$$n_{\text{over}} \geq m_{\text{eff}}, \qquad m_{\text{eff}} = \begin{cases} \max\big(1, \min(m, n)\big) & \texttt{available}, \\ \max(1, m) & \texttt{fixed}, \end{cases} \tag{11}$$

with $m =$ `network.gross_los_min_rows` (loader default 2) and `network.gross_los_min_rows_rule` (loader default `fixed`, reference value `available`). A gross line of sight is wrong on every code band at once; one heavy-tailed row is one band's multipath. Under `fixed` a line of sight with one code row can never be declared; `available` reaches it and still demands two rows where two exist. Declaring takes `gross_los_epochs` (default 2) consecutive epochs of evidence (absence resets the streak) and passes two counted guards: the maturity horizon (`gross_los_warmup_epochs`, default 120 emitted epochs) and the whole-station stand-down (`gross_los_max_station_frac`, default 0.5: a station-wide fault is not a line-of-sight fault). Release takes every code row under `gross_los_clear_sigma` (3) for `gross_los_clear_epochs` (2) epochs and is never guarded. A declaration unseen beyond `gross_los_max_gap_s` (900 s) expires with its own event: the next sighting is a new arc.

The cold bound. The maturity horizon holds every declaration above for the first `gross_los_warmup_epochs` emitted epochs, because a cold post-fit code residual measures the unconverged station block. A faulty receiver channel inside that window is a different case: its carried ambiguity makes the line of sight redundant, and the fault then enters the satellite clock at metres per epoch, which the dense border carry keeps. With `network.gross_los_cold_bound_m` $= b$ (loader default 0, off; reference value 12 m) a line of sight whose largest post-fit code residual $|v|$ exceeds b inside the horizon is declared at once: one epoch, no row count, no persistence. The same whole-station stand-down applies (more than `gross_los_max_station_frac` of a station's lines of sight over the bound declares none). While a cold declaration stands, every epoch on which $|v| > b$ again erases the carried ambiguity again, because a free ambiguity absorbs its phase row exactly whatever the

number of bands; at the release and at an expiry the ambiguity is erased once more, so the line of sight reconnects on a fresh ambiguity and not on one harvested while the fault was in its rows. The cold path's edges are counted apart from the sigma-bar path's.

Response. From the next assembly the declared code rows fold at $\sigma\times$`gross_los_deweight` (default 100; `networkGrossLosPublishScales`); the row keeps its sign and still anchors the float if the phase side disappears. At the declaration the line of sight's leveling-carry entries (`AmbGrossErase`, with a pending-reseed mark) and slant-ionosphere entries (`IonoGrossErase`) are erased. Both sites are declared EraseAndRebirth on a decouple event: this response erases where the slip responses drop. Phase rows keep their weight; they have their own tiers. The bounded variant `gross_los_bound` (freeze instead of erase) was falsified and stays off (`docs/refound/RF-3_code_row_qc_2026-09-22.md`).

Two halves and the emission exclusion. The erase must follow the carry harvest (`networkLeveling`), or the harvest re-stashes the erased entries; the decision needs only the solved epoch. With `network.gross_los_emit_exclude` the seam is split (`GrossLosPhase`): the Decide half runs immediately before `networkSolPrep`, the Act half after `networkPostFitRaim`. Unsplit, a declaration reaches the emitted product one epoch late. The emitted clock and code bias are joint-solve states, and removing a station's contribution would take a second solve, so the exclusion acts on the records. For each satellite i screened at n_i stations, e_i of them with a current declaration on it,

$$n_i - e_i < N_{\min} \;\Rightarrow\; \text{withhold } i; \qquad \text{otherwise emit } i \text{ with a discontinuity mark,} \tag{12}$$

with $N_{\min}$ = `network.gross_los_sat_min_stations` (loader default 3). The withhold is per satellite and the epoch still emits. The mark joins the retro queue, whose drain this setting arms, and lands with full channel scope (clock, code bias, phase bias) in the first emission containing the satellite, normally the declaring epoch.

1.7 The emitted clock

The emitted satellite clock c^i is the joint-solve column of the Network-mode document, carried between epochs through the guarded temporal anchor of the float-ambiguity-vehicle section. Four output-only steps act on it between the solve and the product; none changes the solve.

Held frame (`network.emit_clock_frame_hold`). The common mode of the satellite clocks of one constellation is the clock-ensemble gauge: a user's receiver clock absorbs it identically on code and phase. The solve re-realises it freely, so the raw emitted clock moves by a common amount from epoch to epoch that carries no physical content (`docs/SPEC_clock_emit_rebase.md`). For constellation k let $c^i_{\text{held}}(t^-)$ be the previous emitted value and A_k the frame offset. The median innovation of the fresh, unforced keys (at least `min_common` = 4 of them),

$$\mu_k(t) = \operatorname*{med}_i \Big[\big(c^i(t) - A_k\big) - c^i_{\text{held}}(t^-) \Big], \qquad A_k \leftarrow A_k + \mu_k(t), \qquad c^i_{\text{held}}(t) = c^i(t) - A_k, \tag{13}$$

is accumulated, and every satellite, a new sighting included, is emitted in the held frame. One subtrahend serves the constellation, so between-satellite differences are unchanged. When $|A_k|$ exceeds `network.clock_frame_leash_m` (default 50 m) the constellation re-anchors to the solve frame, and the re-anchor is a datum mark so the re-coupling edge is declared.

Clock step test (`network.clock_disc_jump_test`). After the frame hold, a satellite whose common-removed step

$$d^i(t) = c^i_{\text{held}}(t) - c^i_{\text{held}}(t^-) \tag{14}$$

(with the hold off, the raw step less its median μ_k) exceeds `network.clock_disc_jump_m` (default 1.5 m, a flat threshold set well above the per-epoch walk step) is reported for a retro-bump: an `R` record names the satellite, the satellite's discontinuity counters step, and the user resets that satellite's ambiguities at the event epoch. Keys already marked in the epoch (a pending retro-bump, a datum-changed constellation) are excluded from the vote and from the test, so a marked event neither drags the frame nor is marked twice. A satellite returning after more than `max_gap_s` is a first sighting and is not tested unless the gap-aware extension (`network.clock_disc_gap_jump_m`, default 0, off) tests it against its retained pre-gap baseline. The clock counter of the emitter has no magnitude test of its own; the step test lives at the emission-preparation tier and drives the forced path, which a replay reproduces from the serialised `R` records. A clock step that reaches the product without being declared is a contract violation, so the test must be armed (`docs/cards/clock/CS-clock-step-signalling.md`).

Clock discontinuity counter (`network.emit_clock_disc`). With the setting on, a clock-pivot handover bumps the clock counters of the changed constellations only and serialises that constellation set on the epoch record; with it off the bump is global. The setting is part of the contract (Section 1.9): a replay emitter built without it misses the scoped declarations.

Quality channel (`network.emit_ura`). Each emitted clock with a non-pinned column carries

$$\text{URA}^i = \max\{1,\ \bar{\sigma}_0\}\,\sqrt{\boldsymbol{P}_{cc}}, \tag{15}$$

with $\boldsymbol{P}_{cc}$ the posterior variance of the satellite's shared clock column (a pinned clock has an artificial variance and no URA) and $\bar{\sigma}_0$ the exponentially weighted variance factor of post-fit RAIM (`network.ura_ewma_alpha`, default 0.1; one epoch behind, because the post-fit tier runs after the emission). A satellite refloated within the inflation horizon is multiplied by `network.ura_refloat_inflate` (default 1, no inflation). The channel needs `network.raim`; without it the URA is the formal clock sigma alone and the loader warns.

Neighbouring checks. The clock runaway check (`network.clock_runaway_check`, armed) drops from the emission a constellation whose median clock differs from the global median by more than `network.clock_runaway_mag_m` (1000 m) before the product is written; the drift ceiling needs the two-state clock model, which is off (the emission-gate section). The anchor's decoupling and re-coupling edges and its marks are described with the anchor itself; a mark names the satellite and steps its counters, so the user re-floats exactly that satellite.

1.8 The emitted phase bias

For satellite i, signal s and emission epoch t, let $c^i(t)$ be the emitted clock (C_0, metres) and $p^i_s(t)$ the estimator's phase bias in the emit frame (the `bias_m` `networkPbHold` receives). The product carries $C^i = c^i$ and P^i_s; the user applies $-C$ on code and $-C + P$ on phase (the SSR phase-bias equation).

The generation hold. Under `network.emit_pb_hold` each key (i, s) carries a generation $g \geq 1$, started at t_0 when the key is first emitted or was last emitted more than T_{h} ago (`network.emit_pb_hold_max_gap_s`, loader default 300 s; not set by the reference configurations). Within a generation

$$P_s^i(t) = p_s^i(t_0). \tag{16}$$

The generation is declared by a `gen=`g token on every P record, by `SsrPhaseBiasEntry::hold_generation` in the in-process store, and by `pbhold=` on the S line (1 plain, 2 phase-clock mode, 3 smoothed). The solve and the C and B records are byte-identical with the setting on. The premise, graded against truth (`docs/refound/PBH_phase_bias_hold_2026-09-19.md`): the phase-bias motion within an arc is error that adds to the clock error. Like the `iode=` cutover, the contract states that the value has not changed and names the epoch at which that statement restarts; handover steps of the estimate stay out of the emitted product.

Ordering contract. `networkPbHold` is the last writer of `bias_m` in `networkEmitStore`: after the carry-invariant emission re-base (gated on `network.carry_invariant`, inert on the reference configurations) and after every discontinuity-marking site; before the emitter's value-jump test, which then screens the held value; before the file write, which reads the same solution as the in-process store.

Both edges. Decoupling: each held emission records $|p_s^i(t) - P_s^i(t)|$ into a run-end distribution. Re-coupling: each generation start is a counted `PB_HOLD` `gen_start` event (cause `first` or `gap`). A mark naming a satellite's clock and phase bias together states that both moved; the plain hold drops the phase-bias half and records the de-commoned clock step it left out (the phase-clock mode below transfers it). The de-commoned step of a satellite is its emitted clock step less the median step of its own constellation, $d^i = [c^i(t) - c^i(t^-)] - M_{k(i)}$, so a common mode that differs between constellations is not charged to either.

The phase-clock mode. Under `network.emit_pb_hold_phase_clock` (armed) the common clock motion of constellation k accumulates over consecutive emissions $t^- < t$,

$$M_k(t) = M_k(t^-) + \operatorname*{med}_{j \in \mathcal{S}_k(t)} \left[c^j(t) - c^j(t^-)\right], \tag{17}$$

over the satellites $\mathcal{S}_k(t)$ of k with a finite clock at both emissions (an even count takes the mean of the two middle values; `pbHoldCommonClockStep`). With $\tilde{c}^i(t) = c^i(t) - M_{k(i)}(t)$ the held emission is

$$\boxed{P_s^i(t) = p_s^i(t_0) + \left[\tilde{c}^i(t) - \tilde{c}^i(t_0)\right].} \tag{18}$$

An emission without the satellite's clock reuses the last transfer; a generation started without one is held plainly. Subtracting from $C^i = c^i$ gives the phase-side identity

$$C^i(t) - P_s^i(t) = c^i(t) - p_s^i(t_0) - c^i(t) + M_k(t) + c^i(t_0) - M_k(t_0) = \left[c^i - p_s^i\right](t_0) + M_k(t) - M_k(t_0). \tag{19}$$

The constant goes into the user's float ambiguity. The common mode is part of every c^j of the constellation, so it enters code and phase rows alike and the receiver clock and inter-system bias absorb it. Satellite-specific clock motion reaches the code rows only. In the family $(C - P)(t) = (c - p)(t_0) + \Phi(t)$ the phase-clock mode is $\Phi = M_k(t) - M_k(t_0)$; the plain hold is $\Phi = c(t) - c(t_0)$, every clock move on the phase side; no hold passes the estimator's own phase-visible change, wander included.

Why the mode exists. Under the exact coupling time update armed on the reference configurations (`network.carry_coupling_exact_update`; the float-ambiguity-vehicle section) the satellite's level converges: the estimator moves c^i and every p_s^i by the same amount, invisible to phase. The plain hold freezes P while C follows, turning the convergence into a phase-side drift the user's float ambiguities cannot follow. Under the phase-clock mode the move shifts $\tilde{c}^i$, hence P, by the same amount (unless common to the constellation): $C - P$ stays and the level reaches the code side only (`docs/refound/LAH_level_aware_hold_2026-09-29.md`, `EXL_exact_update_lah_promotion_2026-10-01.md`).

Limitation. The mode is interim. Within a generation the phase-visible clock motion the user sees is the a priori clock product's alone, since the emitted clock refines the a priori clock (the closed-loop section) and the hold passes none of the network's satellite-specific motion to the phase side. That is valid with a precise a priori clock (`network.apriori_source sp3`, the default). The loader warns under `brdc_rts` and refuses the mode under `sp3_brdc_clock`. The smoothed member (`network.emit_pb_hold_smoothed`, `pbhold=3`), which carries a clock clone, is built and not armed; it excludes the phase-clock mode and needs the exact update and the temporal anchor.

Remark 1. A datum move between two generation starts, such as a phase-anchor handover with its S-transform, moves the estimator's phase biases and not the held values (counted, measure-only: `prm_pbhold_unmoved_at_shift`). A float user absorbs each held constant in its own ambiguity. An integer-resolving user needs every phase bias of a constellation in one S-basis (the explicit phase-bias section; [1]), which a held product does not guarantee after a datum move. The reference configurations run the test station's user float (`closed_loop.ar_mode` 0).

1.9 The product contract

The emitted product is a line-based file read by the replay driver and the real-time user, and pushed record by record into an in-process store for the test station. Each epoch is an `E` line (week, time of week, IOD, provider, solution identifier, the datum-change flag) followed by its `C` (clock, with the counters and the URA), `B` (code bias) and `P` (phase bias with the signal-integer flags, the counter and the `gen=` token) records; `R` records list the retro-bumped satellites, `I` the solution-generation identifier, and `H` (`network.emit_epoch_status_record`, armed) an epoch-status heartbeat that also stands alone for a withheld epoch. Optional records and tokens appear only when their setting is on, so a run with them off writes the base format.

The S line. The header and any periodic re-broadcast (`network.contract_rebroadcast_s`) carry a machine-readable contract (`SbasisContract`). It declares the reference receiver, the clock pivot of each constellation, the ambiguity datum, the inter-system datum, the code-bias datum and the bias sign; the wind-up construction and mode (`wumode=`); the identity of the a priori orbit and clock product and of the frame, coordinate epoch, antenna file and orbit reference point; the code-bias pin pair of each constellation and the value of the geometry-free datum (`sat_gf_datum=`); and the parameters with which a replay emitter reproduces the discontinuity counters: the phase-bias discontinuity threshold (flat in metres, or a fraction of the wavelength through `discfrac=`, set by `network.disc_threshold_frac_cycles`, reference value 0.25), the clock-counter channel (`emit_clock_disc`), the staleness horizon, the code-bias jump test (`cbjump=`), the clock set-change trigger (`clkset=`) and the mark scope (`discscope=`). The token `pbhold=` (hold, phase-clock mode, smoothed mode) states how the phase-bias channel is read, together with the `gen=` token on every `P` record, and `gid=` names the run that wrote the product.

The check. `NetworkProductReader::checkContract` compares a contract read from a file with the contract a user expects and lists every mismatch. The comparison covers the reference receiver, the bias sign, the wind-up construction, the ambiguity, inter-system and code-bias datums, the code-bias emission flag, the pivot of each constellation the user declares, the pin pairs of each declared constellation, the seven replay-emitter parameters, the three hold flags, the a priori product, and the identity fields and generation identifier the user declares. Two rules matter for interpretation. The wind-up mode is compared only when both sides declare one, because a product written before the declaration cannot be verified and keeps replaying; a mismatch means a different carrier-phase frame, by whole cycles per satellite and signal, which a float ambiguity absorbs without any residual, so the check is the only place the difference is visible. The hold flags are compared in both directions, because a `gen=` payload under a contract that declares no hold (or the reverse) means the contract and the records disagree. A product without an S line is refused by the replay unless `network.replay_allow_undeclared_wire` is set, since the replay emitter would otherwise be built with defaults and miss every clock declaration the network made.

Self-check. At the end of a run the writer is closed, the product is read back, and the parsed contract is checked against the one written, together with the realised pivot map and, when re-broadcast is armed, every re-broadcast receipt. The outcome is written to `network_contract_check.csv`; a run that emitted nothing records a failure row instead of leaving the file absent, so a missing file means only that the check never ran.

1.10 The user's answer

The in-process user of the test station (undifferenced in the reference configurations) reacts in `realtime/RealTimeEngineSsrBreaks`, in a fixed order. First, `reactToGenerationChange` reads the product's solution-generation identifier, which is distinct from the hold generation; a change (a restart of the network, a global re-realisation) resets every network-corrected ambiguity and re-inflates the receiver code-bias states. It runs before the per-signal reactions because a restart re-seeds the discontinuity counters. The order of the phase-bias reactions that follow, with the translation alternatives, is also stated in the phase-bias step section of the SSR document; this section gives the answer to the hold and to its presence edges.

Hold generation. `resetOnPhaseBiasDisc` dispatches `reactToPbHold_` before any other phase-bias reaction, keyed on the data: a key whose store entry has a nonzero `hold_generation` is claimed. Its discontinuity counter is consumed, so the per-key memory advances, and ignored, since under a hold a counter change carries no value step. `PbHoldLedger::react` decides: *FirstSight* adopts the generation, *Held* keeps it, *Refloat* answers a change of a previously seen generation with `StateManager::resetAmbiguity` at the configured ambiguity σ, the call a discontinuity reset uses. First sightings, held cells and refloats sum to the visited cells.

Presence edges. Under the bias staleness gate (`processing.enable_ssr_bias_max_age`, armed on the derived user of the closed loop) the user tracks

$$a_s(t) = \big[\text{bias present} \wedge \text{age} \le \texttt{ssr_bias_max_age_s} \cdot \texttt{ssr_bias_stale_hold_factor}\big] \qquad (20)$$

(hold factor default 2) and answers an edge $a_s(t) \neq a_s(t^-)$, the freshness flip, with an ambiguity reset. A held product has two such edges with no generation change: a held bias appearing on a phase row already in use without it, and one leaving such a row; either steps the applied correction

by the whole bias. Without the setting below, a claimed key skipped every downstream reaction, the freshness flip included, and its memory was not written while held, so both edges went unanswered (a first sighting lacks prior state only on a new row). The update's post-fit peel then attributed the residual to other satellites, reset their ambiguities and could exclude the faulty satellite's code rows; one real-data epoch moved the position by about 490 m (`docs/refound/PBF_pb_hold_presence_edges_2026-09-28.md`). With `processing.ssr_pb_hold_presence_flip`, derived from `network.closed_loop.pb_hold_presence_flip`, a claimed key skips only the counter reaction, which the hold owns, and its freshness edge runs as on an unheld key: on the reference configurations a reset, since the step translation (`closed_loop.pb_step_translate`) and the level check (`closed_loop.amb_level_check`, harmful under the hold) are off. When the network declares a signal roster and the user latches on it (`network.closed_loop.roster_latch`), a flip on a signal for which the network declares no phase-bias family is attributed to an absent family and not to staleness, and the ambiguity is left alone.

Translation instead of reset. Two optional reactions replace the reset by an exact translation of the float ambiguity by the step of the applied phase bias: the declared-step carry (`processing.ssr_phase_bias_disc_carry`) and the step translation (`closed_loop.pb_step_translate`, which also answers undeclared steps). Both are off on the reference configurations. Their algebra, their order against the hold and the reset, and their fallbacks are in the phase-bias step section of the SSR document; the derived configuration of the in-process user is in the section on the user of the network's corrections.

1.11 Status

Armed on both reference configurations through the promotion ledger (`docs/CONFIG_promotion_ledger.json`): gap re-entry per row (`docs/cards/slip/CARD-G-GAP-REENTRY-RELEVEL.md`), per signal (`docs/refound/GRS_gap_reentry_per_signal_2026-09-20.md`) and with the tracker's band naming (`GRK_gap_reentry_key_mismatch_2026-09-22.md`); the MW re-flag skip; the re-admission horizon and its mirrors (`docs/cards/network-vehicle/READMIT-EDGE.md`); accumulated wind-up with its restart response; the gross line-of-sight screen with its cold bound; the generation hold in its phase-clock mode; the held clock frame, the clock step test, the clock discontinuity counter and the quality channel; the user's presence edges. The baseline with these settings (`docs/refound/BOF_built_off_follow_through_2026-10-28.md`), over a 6 h run: the test station WTZZ00DEU on real data, median 3-D position error per hour 0.059, 0.012, 0.023, 0.012, 0.017 and 0.012 m; the test station SZZ000SIM on the simulated network, graded from the run start, 0.137, 0.072, 0.031, 0.024, 0.026 and 0.018 m. Not armed: the co-resumed share, which answers only the third-frequency case it was built for (`docs/CONFIG_promotion_ledger.json`); the wind-up gap bridge; the smoothed hold; the user's translation reactions.

Open: the hold in the warm-resume snapshot and the cause of the phase-bias wander; the held emission under a phase-bias shift; the regressions of the phase-clock hold at stations tracking the C1W signal and its need for a precise a priori clock, which the self-contained clock level was to remove and did not; a dedicated emitted-product writer for the gross screen; the failed real-data triple-frequency bar of the per-signal tier; and the peel's masking of a bad phase row. The register is `docs/status/OPEN_ITEMS.md`.

References

[1] D. Odijk et al. “On the estimability of parameters in undifferenced, uncombined GNSS network and PPP-RTK user models by means of S-system theory”. In: *Journal of Geodesy* 90.1 (2016), pp. 15–44.